

Requisitos de programacao

Explicacao direta de como programar insercao no inicio, meio e fim, e como a busca se comporta nesses mesmos pontos.

Estrutura base

A lista guarda uma referencia para o primeiro no, chamada inicio. Opcionalmente tambem pode guardar fim e tamanho.

Operacoes

Inserir no inicio, inserir no meio, inserir no fim e buscar valores percorrendo os nos.

Regra geral

Como cada no aponta so para o proximo, muitas operacoes precisam caminhar desde o inicio.

No, inicio e fim

Antes das operacoes, a lista precisa ter uma classe No e referencias para controlar

a

```
public class No {  
    public int valor;  
    public No prox;  
  
    public No(int valor) {  
        this.valor = valor;  
        this.prox = null;  
    }  
}  
  
No inicio = null;  
No fim = null;  
int tamanho = 0;
```

Como ler

inicio aponta para o primeiro no.
fim aponta para o ultimo. tamanho
ajuda a validar posicoes no meio da
lista.

Novo no vira o primeiro

Passos

1. Criar o novo no.
2. novo.prox recebe o antigo inicio.
3. inicio passa a ser o novo no.

```
public void InserirInicio(int valor) {  
    No novo = new No(valor);  
    novo.prox = inicio;  
    inicio = novo;  
  
    if (fim == null) fim = novo;  
    tamanho++;  
}
```



Precisa parar no no anterior

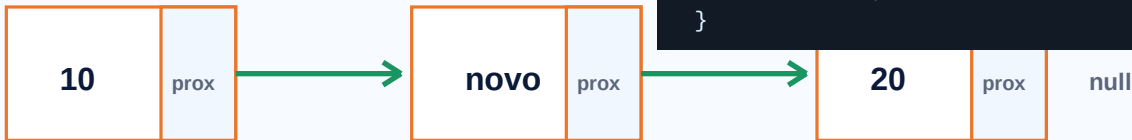
Passos

1. Validar a posicao.
2. Caminhar ate posicao - 1.
3. novo.prox aponta para anterior.prox.
4. anterior.prox aponta para novo.

```
public void InserirNaPosicao(int valor, int pos) {  
    if (pos == 0) { InserirInicio(valor); return; }  
    if (pos == tamanho) { InserirFim(valor); return; }
```

```
    No anterior = inicio;  
    for (int i = 0; i < pos - 1; i++)  
        anterior = anterior.prox;
```

```
    No novo = new No(valor);  
    novo.prox = anterior.prox;  
    anterior.prox = novo;  
    tamanho++;  
}
```



Novo no vira o ultimo

Passos

1. Criar o novo no.
2. fim.prox aponta para novo.
3. fim passa a ser novo.
4. Se a lista estava vazia, inicio tambem recebe novo.



```
public void InserirFim(int valor) {  
    No novo = new No(valor);  
  
    if (inicio == null) {  
        inicio = novo;  
        fim = novo;  
    } else {  
        fim.prox = novo;  
        fim = novo;  
    }  
    tamanho++;  
}
```

Uma funcao resolve inicio, meio e fim

A busca sempre compara o valor do no atual. Se nao achou, avanca para o proximo.

```
public No Buscar(int valor) {  
    No atual = inicio;  
  
    while (atual != null) {  
        if (atual.valor == valor)  
            return atual;  
  
        atual = atual.prox;  
    }  
  
    return null;  
}
```

Ideia

A mesma busca encontra valores no inicio, no meio ou no fim. O que muda e a quantidade de passos ate encontrar.

Melhor caso

Como funciona

1. atual começa em inicio.
2. atual.valor ja e o valor procurado.
3. A funcao retorna o no imediatamente.

Custo

$O(1)$ porque nao precisa andar pela lista.



Percurso parcial

Como funciona

1. Compara com o primeiro no.
2. Não encontrou, então $atual = atual.prox$.
3. Repete até chegar no no do meio.
4. Ao encontrar, retorna o no.

Custo

$O(k)$, onde k é a posição do valor encontrado.



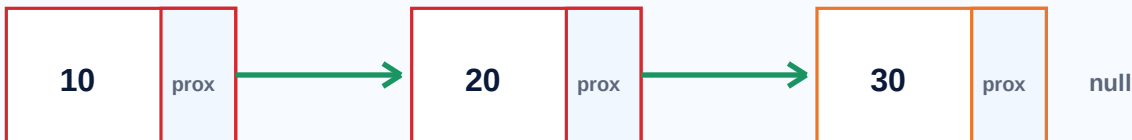
Pior caso encontrado

Como funciona

- Q
1. Compara desde o início.
 2. Avanca no por no.
 3. So encontra no ultimo elemento.
 4. Depois retorna o no encontrado.

Custo

$O(n)$, porque pode precisar visitar todos os nos.



O que muda entre inicio, meio e fim

O segredo e saber quando precisa percorrer a lista e quando basta mexer nas referencias diretas.

Inicio

Insercao e busca no inicio sao rapidas. O inicio ja aponta para o primeiro no.

Meio

Normalmente precisa percorrer ate o ponto certo antes de inserir ou encontrar.

Fim

Inserir no fim fica rapido se existir ponteiro fim. Buscar no fim continua exigindo percurso.